11 4 182

track_branch_label.sh — §11.4.182 track+branch+alias+model+effort work-stream label

Revision: 2 **Last modified:** 2026-09-26T14:40:00Z **Authority:** constitution submodule §11.4.182 (Track+branch work-stream identity label + the 2026-07-26 `<model>` / `<effort>` EXTENSION) · §11.4.231 clause (F) (effort-tier) · §11.4.178 (track-qualified identity) · §11.4.177 (inherited-by-reference) · §11.4.6 (no-guessing / honest boundary) **Classification:** universal (§11.4.17)

Overview

`constitution/scripts/multitrack/track_branch_label.sh` is the **reference labeler** — the single source of truth for the §11.4.182 work-stream identity label. It prints, for the CURRENT checkout + live session, the canonical prefix that every agent / subagent / work-stream label AND every operator-facing work-stream reference MUST start with, so parallel-track work (`/mnt/track1..4`) is never ambiguous.

The §11.4.182 guard hook (`guard-track-branch-label.sh`) calls this labeler to derive the live alias for its alias-correctness check and its "correct label" block message (DRY — the derivation lives in ONE place).

The current (2026-07-26 EXTENSION) output is the **5-field** form:

```
(T<N>/<branch> - <alias> - <model> - <effort>)
```

The earlier 3-field `(T<N>/<branch> - <alias>)` and 4-field `(T<N>/<branch> - <alias> - <model>)` forms remain valid and are still accepted by the guard during migration; this labeler emits the full 5-field form.

Fields (all deterministic; every `?` fallback is honest, never guessed — § . .)

Field	Source	`?` when
`<N>`	cwd path `/mnt/track<N>/...`	cwd not under `/mnt/trackN`, or the token is non-numeric
`<branch>`	`git rev-parse --abbrev-ref HEAD`	not a git repo (`HEAD` on de-tached HEAD)
`<alias>`	`CLAUDE_CONFIG_DIR` basename `.claude-<alias>` → `<alias>`	`CLAUDE_CONFIG_DIR` unset / non-`.claude-*`
`<model>`	model currently answering for `<alias>` — see below	derivation source unreadable / absent
`<effort>`	live reasoning-effort signal, closed ladder `{low|medium|high|xhigh|max}`	no signal exposed, or a present value outside the lad-der

`<model>` derivation — a closed two-case split on the shape of `<alias>`

- **CASE A — native alias** (`<alias>` matches `claude[0-9]*`, e.g. `claude1..claude5`): native aliases talk to the real Anthropic API with NO fixed model — Claude Code itself chooses/switches the serving model per turn and does not expose that choice via any env var. The only live source is the CURRENT session's OWN transcript at `$CLAUDE_CONFIG_DIR/projects/<slug>/$CLAUDE_CODE_SESSION_ID.jsonl` (where `<slug>` is the cwd with every non-alphanumeric char replaced by `-`, the exact project-slug algorithm the Claude Code CLI itself uses). The LAST `"type":"assistant"` entry's `message.model` is mapped to a short native name by case-insensitive substring match against `{opus, sonnet, haiku, fable}`. An id matching none of the four is printed VERBATIM (real-but-unrecognised is never silently dropped to `?`). Missing session-id / config-dir / an unreadable or model-less transcript ⇒ `<model>` is `?`.

- **CASE B — provider alias** (everything else, e.g. `deepseek`, `xiaomi`, `kimi-for-coding`, `opencode`, or a raw `prov-<id>` basename): the real serving model is HOST CONFIG DATA, read directly from `$HOME/.local/share/claude-multi-account/providers/<id>.env` as `CMA_PROVIDER_MODEL='<real-model-id>'` — the single source of truth the claude-multi-account launcher itself exports at launch (no duplicated model list). A leading `prov-` is stripped and retried once. Unresolvable ⇒ `?`.

Consistency (§11.4.182 EXTENSION clause (3)) is by construction: the native model is read from THAT alias's own live transcript and the provider model from THAT provider's own registry entry — so the derived `<model>` definitionally belongs to the shown `<alias>`; a model that does not belong to the alias is never emitted.

`<effort>` derivation — precedence-ordered, first-present-authoritative

`<effort>` is read from the first PRESENT (non-empty) of the precedence-ordered candidate env vars `CLAUDE_EFFORT` → `CLAUDE_CODE_EFFORT` → `AGENT_EFFORT` → `CMA_EFFORT` (they name the SAME effort signal). The first present candidate is **authoritative**: an in-ladder value is used, and a present value OUTSIDE the closed ladder yields the honest `?` (§11.4.182 EXTENSION clause (7)(a) / §11.4.231 clause (F) — an out-of-set value never falls through to a lower-precedence var, which would mask a misconfigured effort behind a different env var). No candidate present ⇒ `?`.

HONEST HARNESS BOUNDARY (§11.4.6 — stated, never overstated): effort-tiering is REAL where the execution path exposes an effort control and a DOCUMENTED capability-gap where it does not. The current Claude Code **Agent tool** (subagent dispatch) exposes a `model` parameter but **NO `effort` parameter**, so a subagent's effort is NOT settable via that path and its `<effort>` field degrades honestly to `?`. The **Workflow tool's `agent()`** DOES take an `effort` argument, so effort IS settable there. The label reports whichever path is live, never invents one — the same aspirational-vs-harness posture the §11.4.209 / §11.4.211 Opus-`xhigh` review/merge pins occupy.

Prerequisites

- `bash` (`#!/usr/bin/env bash`).
- `git` (optional — its absence yields `<branch>=?` , not an error).
- `sed` (used by the `?` -safe model/effort parsing).
- `jq` (OPTIONAL — used when present for a structural, order-independent parse of the native-alias transcript; its absence degrades to a documented line-anchored `grep` / `sed` fallback, never a failure).
- `CLAUDE_CONFIG_DIR` (optional — its absence/non-match yields `<alias>=?` and, in turn, `<model>=?`).
- `CLAUDE_CODE_SESSION_ID` (optional — its absence yields `<model>=?` for a native alias).

Usage

```
bash constitution/scripts/multitrack/track_branch_label.sh
# -> e.g. (T1/main - claude1 - opus - high)
```

Run from within the checkout whose label you want. Read-only; §11.4.128-safe (the native-alias lookup reads at most the last 500 lines of the transcript).

Edge cases

Situation	`<model>` / `<effort>`
native alias, transcript readable, last assistant model `claude-opus-4-8`	`<model>=opus`
native alias, missing `CLAUDE_CODE_SESSION_ID` / unreadable transcript	`<model>=?`
provider alias `deepseek` with a resolvable `deepseek.env`	`<model>=<CMA_PROVIDER_MODEL>`
`CLAUDE_EFFORT=high`	`<effort>=high`
`CLAUDE_EFFORT=garbage` (present, out-of-ladder)	`<effort>=?` (no fall-through)
`CLAUDE_EFFORT` empty, `CLAUDE_CODE_EFFORT=xhigh`	`<effort>=xhigh`
no effort env var set (e.g. Agent-tool subagent)	`<effort>=?` (documented harness gap)

11 4 6

The script exits `0` in all of these — a `?` field is a valid honest label, not an error.

Honest limitations (§ . . — stated, never hidden)

- **Subagent self-introspection of `<model>`.** The native-alias transcript lookup is proven reliable when invoked from the DISPATCHING process's own shell (the guard hook's actual execution context at dispatch time). Invoked from INSIDE an already-dispatched background subagent it was observed to return the DISPATCHING conductor's model — the two processes share the same `$CLAUDE_CODE_SESSION_ID` / transcript file on this host's multitrack ruler (§11.4.187) with no `isSidechain` marker distinguishing them. An open, honestly-scoped follow-up (see `docs/research/model_in_label_20260726/DESIGN.md`), never silently "fixed" by guessing.

- `<effort>` **on the Agent tool.** Not settable via the Agent-tool subagent dispatch path (no `effort` parameter) — degrades to `?` per the harness boundary above.

11 4 227

Divergent-labeler note (§ . . EXTENSION clause ( ) / § . .)

This constitution copy carries the full 5-field (model + effort) form. The top-level project duplicate `scripts/multitrack/track_branch_label.sh` currently carries the 4-field (model, no effort) form — functionally equivalent on the alias+model fields, diverging on the effort field. Reconciling the two copies to a single source of truth (the top-level a thin wrapper exec-ing this constitution copy, §11.4.177 inherited-by-reference) is a standing to-be-tracked follow-up, never a silent divergence.

Related scripts

- `constitution/scripts/hooks/guard-track-branch-label.sh` — the §11.4.182 PreToolUse guard that calls this labeler (companion doc: `constitution/docs/scripts/guard-track-branch-label.md`).
- `constitution/scripts/hooks/guard-work-track-binding.sh` — the §11.4.191 work→track binding guard (sibling PreToolUse hook).

Constitutional cross-references

§11.4.182 (track+branch+alias+model+effort label) · §11.4.231 clause (F) (effort-tier) · §11.4.178 (track-qualified identity) · §11.4.187 (multitrack engine) · §11.4.29 (naming) · §11.4.75 (mechanical enforcement) · §11.4.6 (no-guessing) · §11.4.18 (script documentation).

Last verified: 2026-07-26 (Rev 1 — created alongside the §11.4.182 EXTENSION 5-field labeler; effort-loop behaviour re-verified on this checkout: first present candidate authoritative, out-of-ladder → `?`, no fall-through).